# Cifrado

En esta práctica se estudiará cómo realizar una implementación segura de cifrado y descifrado, identificando errores habituales relacionados con el modo de operación, el IV/nonce, el padding, la integridad y la gestión de claves.

Se utilizará Python y se podrá escoger el entorno de desarrollo.

1) Dado el siguiente código:

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

key = b'1234567890123456'
iv = b'0000000000000000'

def encrypt(msg):
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    encryptor = cipher.encryptor()

    padded = msg + b' ' * (16 - len(msg) % 16)
    return encryptor.update(padded) + encryptor.finalize()

def decrypt(ct):
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    decryptor = cipher.decryptor()

    return decryptor.update(ct) + decryptor.finalize()

msg1 = b"Transfer 100 euros from Ana to Juan"
msg2 = b"Transfer 100 euros from Ana to Luis"
c1 = encrypt(msg1)
c2 = encrypt(msg2)
print(c1)
print(c2)
```

Realice lo siguiente y responda a las preguntas planteadas:

a) Ejecute el código varias veces y observe los resultados.

b) Compare el primer bloque (16 bytes) de c1 y c2. ¿Qué observa y por qué ocurre?

c) Identifique al menos tres problemas de seguridad del ejemplo. Considere la clave, el IV, el padding y la ausencia de autenticación/integridad.

2) Dado el siguiente código:

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
key = b'1234567890123456'
def encrypt(msg):
  cipher = Cipher(algorithms.AES(key), modes.ECB())
  encryptor = cipher.encryptor()
  if len(msg) % 16 != 0: raise ValueError("El mensaje debe ocupar bloques completos")
  return encryptor.update(msg) + encryptor.finalize()
```

Realice lo siguiente y responda a las preguntas planteadas:

a) Compruebe si los dos bloques de texto cifrado son iguales. ¿Qué propiedad de ECB explica el resultado?

b) Explique por qué esta propiedad puede filtrar estructura o patrones del mensaje y por qué se considera un problema grave.

3) Dado el siguiente código:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
msg = b"Transfer 100 euros"
ct1 = f.encrypt(msg)
ct2 = f.encrypt(msg)
print(ct1)
print(ct2)
pt = f.decrypt(ct1)
print(pt)
```

Realice lo siguiente y responda a las preguntas planteadas:

a) Ejecute varias veces con el mismo mensaje

b) Compare textos cifrados

c) ¿El texto cifrado cambia en cada ejecución, aunque el mensaje y la clave sean los mismos? ¿Por qué?

d) ¿Qué propiedades de seguridad ofrece Fernet frente al ejemplo de CBC de la Parte 1?

e) ¿Qué mecanismos incorpora Fernet por defecto que en la Parte 1 tendrían que implementarse explícitamente?

f) ¿Qué primitivas criptográficas utiliza Fernet internamente y cómo las combina? ¿Es equivalente a usar un modo AEAD como AES-GCM? Justifique brevemente.

4) Modifique en el código anterior la llamada a decrypt de la siguiente forma:

```
pt = f.decrypt(ct, ttl=60)
```

a) ¿Qué significa ttl? ¿Qué ocurre con un token demasiado antiguo? ¿Y si el token ha sido modificado?

5) Como alternativa moderna, utilice AES-GCM con la clase AESGCM:

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

key = AESGCM.generate_key(bit_length=128)
aesgcm = AESGCM(key)
nonce = os.urandom(12)
msg = b"Transfer 100 euros"
aad = b"cabecera"

ct = aesgcm.encrypt(nonce, msg, aad)
pt = aesgcm.decrypt(nonce, ct, aad)
```

a) Ejecute el ejemplo. Modifique después un byte del texto cifrado o del AAD y compruebe qué ocurre al descifrar.

b) ¿Qué propiedades de seguridad proporciona AES-GCM?

c) ¿Por qué no debe reutilizarse un nonce con la misma clave? Compare brevemente AES-GCM con CBC y con Fernet.